Introduction

Project title: SpendWise

Team ID : SWTID-2026-5987
Team Size : 4
Team Leader : Kalaiyarasi S
Team member : Thasmiya G
Team member : Suba Sree M
Team member : Vidhya K

1.Project Overview

"SpendWise" is a common name for fintech projects, but the most prominent modern version is a full-stack MERN (MongoDB, Express, React, Node.js) application designed for personal finance management.

" SpendWise" is a popular name for finance projects, the most established version is a full-stack MERN application (MongoDB, Express, React, Node.js). This project is designed to bridge the gap between simple manual tracking and complex banking apps.

SpendWise is a personal finance management tool that allows users to log transactions, set budgets, and visualize their spending habits. Unlike banking apps that focus purely on digital records, SpendWise often emphasizes total financial clarity—integrating cash, digital spending, and even asset/liability tracking.

2.Project Objectives

The key objectives of the SpendWise backend are centered around data integrity, security, and providing actionable financial intelligence. While the frontend handles the "look," the backend is responsible for the "math" and the "memory."

1. Data Integrity & Management

Centralized Ledger : Provide a single "source of truth" for all income and expenses.

Accurate Math: Use server-side Aggregation Pipelines to calculate balances and totals reliably.

2. Security & Privacy

Identity Protection: Secure user data using JWT (authentication) and Bcrypt (password hashing).

Safe Storage: Ensure financial records are private and protected against unauthorized access or injection attacks.

3. Financial Intelligence

Categorization: Automatically group spending (e.g., Food, Rent) to reveal user habits.

Budget Tracking: Monitor real-time spending against user-defined limits to prevent overspending.

4. Performance & Scalability

Fast Retrieval: Use database indexing so dashboards load instantly, even with thousands of entries.

Extensibility: Build a clean REST API that can easily power web, mobile, or smartwatch apps.

3.Target users

SpendWise targets anyone who needs to bridge the gap between earning money and wondering where it went.

1. Students & Young Adults

Goal: Managing limited allowances and avoiding "month-end" broke cycles.
Need: Simple, fast logging for daily small expenses (coffee, transport).

2. Early-Career Professionals

Goal: Managing their first consistent salary and saving for big goals (travel, gadgets).
Need: Monthly budget limits and "lifestyle creep" tracking.

3. Freelancers & Gig Workers

Goal: Tracking irregular income streams from multiple sources.
Need: Clear visualization of Income vs. Expenses for tax prep and stability.

4. Over-Spenders

Goal: Identifying "leaks" in their budget (unused subscriptions, impulsive buys).
Need: Real-time category breakdowns and over-budget alerts.

4.Pre-requisite & knowledge requirements

1.Core Languages & Runtimes

JavaScript (ES6+): Proficiency in async/await, arrow functions, and array methods (map, filter, reduce).
Node.js: Understanding the event loop and package management via npm or yarn.

2. Frameworks & Tools

Express.js: Knowledge of routing, controllers, and Middleware (essential for auth checks).
Postman/Insomnia: Ability to test API endpoints without a frontend.

3. Database Mastery (MongoDB)

Mongoose ODM: Designing schemas and handling data validation.
Aggregation Framework: Critical for calculating financial totals using $group and $sum.

4. Security Basics

JWT (JSON Web Tokens): For managing secure user sessions.
Bcrypt: For hashing and salting user passwords.
Environment Variables: Using .env files to hide database credentials.

5.Project architecture and tech stack

Project Architecture:

Frontend: User interface for adding and viewing expenses.
Backend: Handles API requests, authentication, and business logic.
Database: Stores user details and transaction data.

Tech Stack:

Frontend: React, Tailwind CSS
Backend: Node.js, Express.js
Database: MongoDB
Authentication: JSON Web Token (JWT)

Architecture Flow:

User → Frontend → Backend → Database.

6.Project setup and configuration

1. Install Node.js and MongoDB.
2. Create project folder and initialize project using npm init.
3. Install required packages: express, mongoose, cors, dotenv, jsonwebtoken, bcrypt.
4. Create folders: models, routes, controllers, config, middleware.
5. Configure database connection using MongoDB.
6. Create .env file for environment variables.

Start the server using npm run dev and test APIs.

7. Features&api endpoints

Features:

1.User Registration & Login
Users can create an account and securely log in to the system.
Authentication is handled using JSON Web Token.

2. Add Income and Expenses

Users can record their daily income and expenses with details such as amount, category, and date.

3. View Transaction History

The system displays all transactions so users can easily track their spending and income records.

4. Edit and Delete Transactions

Users can update incorrect entries or delete transactions that are no longer needed.

5. Expense Tracking and Reports

The application shows summaries or charts to help users understand their spending patterns.

API endpoints:

1.Register API

Allows a new user to create an account by providing name, email, and password

2.Login API

Verifies user credentials and returns a token for secure access.

3.Add Transaction API

Saves new income or expense details to the database.

4. Get Transactions API

Retrieves all transaction records of the logged-in user.

5.Update Transaction API

Allows users to modify existing transaction details.

6.Delete Transaction API

Removes a transaction from the database.

7.User Profile API

Fetches user information such as name and account details.

8.Project flowchart

SPENDWISE
Expense Submission
Employee
ERP
HR
CRM
AP
Integration &
Reimbursement
Approval Workflow
Expense Submission
Expense Reporting
& Analytics

9. Output

le Edit Selection View Go Run Terminal Help

NM

THUNDER CLIENT

New Request

Activity Collections Env

filter activity

POST localhost:5000/api/transactions
just now

POST localhost:5000/api/auth/login
56 mins ago

POST localhost:5000/api/auth/register
1 hour ago

TC New Request TC New Request TC New Request

POST http://localhost:5000/api/auth/register

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

```
1 {
2   "name":"kalaiyarasi s",
3   "email":"kalai@gmail.com",
4   "password":"kl123"
5 }
```

Status: 201 Created Size: 265 Bytes Time: 415 ms

Resp

```
1 {
2   "_id": "69ac429a5549f782a14c16f9",
3   "name": "kalaiyarasi s",
4   "email": "kalai@gmail.com",
5   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
      .eyJpZCI6IjY5YWM0MjlhNTU0OWY3ODJhMTRjMTZmOSIsImlhdCI6MTc3Mjg5NjkyMiwiZXhwIjoxNzczNTAxNzIyfQ
      .I1TasmZb4yEx2TkVqBJU2S5Hwvshkl7oavpKYfB72IY"
6 }
```

Response Ch

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

node - spendwise

```
PS C:\Users\kalaiarasi\OneDrive\Desktop\NM\server\spendwise> npm run dev
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.3.1] injecting env (3) from .env -- tip: enable debug logging with { debug: true }
Server running on port 5000
MongoDB Connected: ac-sg1nthw-shard-00-01.s3mna8f.mongodb.net
JsonWebTokenError: invalid token
    at module.exports [as verify] (C:\Users\kalaiarasi\OneDrive\Desktop\NM\server\spendwise\node_modules\jsonwe
verify.js:82:17)
    at protect (C:\Users\kalaiarasi\OneDrive\Desktop\NM\server\spendwise\middleware\authMiddleware.js:14:27)
    at Layer.handleRequest (C:\Users\kalaiarasi\OneDrive\Desktop\NM\server\spendwise\node_modules\router\lib\la
152:17)
    at next (C:\Users\kalaiarasi\OneDrive\Desktop\NM\server\spendwise\node_modules\router\lib\route.js:157:13)
```

File Edit Selection View Go Run Terminal Help

THUNDER CLIENT

New Request

Activity Collections Env

filter activity

POST localhost:5000/api/transactions
just now

POST localhost:5000/api/auth/login
56 mins ago

POST localhost:5000/api/auth/register
1 hour ago

New Request | New Request | New Request

POST http://localhost:5000/api/auth/login Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

```
{
  "email":"kalai@gmail.com",
  "password":"kl123"
}
```

Status: 200 OK Size: 265 Bytes Time: 259 ms Response


```
{
  "_id": "69ac429a5549f782a14c16f9",
  "name": "kalaiyarasi s",
  "email": "kalai@gmail.com",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
      .eyJpZCI6IjY5YWM0MjlhNTU0OWY3ODJhMTRjMTZmOSIsImlhdCI6MTc3Mjg5NzI1MywiZXhwIjoxNzczNTAyMDUzfQ
      .OuqbzJ74mB3j-REu31Xqval5KhXKSSeK2itrKCFJXoU"
}
```


Response Chart

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS node - spendwise

```
PS C:\Users\kalaiarasi\OneDrive\Desktop\MM\server\spendwise> npm run dev
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
[dotenv@17.3.1] injecting env (3) from .env -- tip: ⚙ enable debug logging with { debug: true }
Server running on port 5000
MongoDB Connected: ac-sg1nthw-shard-00-01.s3mna8f.mongodb.net
JsonWebTokenError: invalid token
    at module.exports [as verify] (C:\Users\kalaiarasi\OneDrive\Desktop\MM\server\spendwise\node_modules\jsonwebtoken\
verify.js:82:17)
    at protect (C:\Users\kalaiarasi\OneDrive\Desktop\MM\server\spendwise\middleware\authMiddleware.js:14:27)
    at Layer.handleRequest (C:\Users\kalaiarasi\OneDrive\Desktop\MM\server\spendwise\node_modules\router\lib\layer.js:
152:17)
    at next (C:\Users\kalaiarasi\OneDrive\Desktop\MM\server\spendwise\node_modules\router\lib\route.js:157:13)
```

CHAT

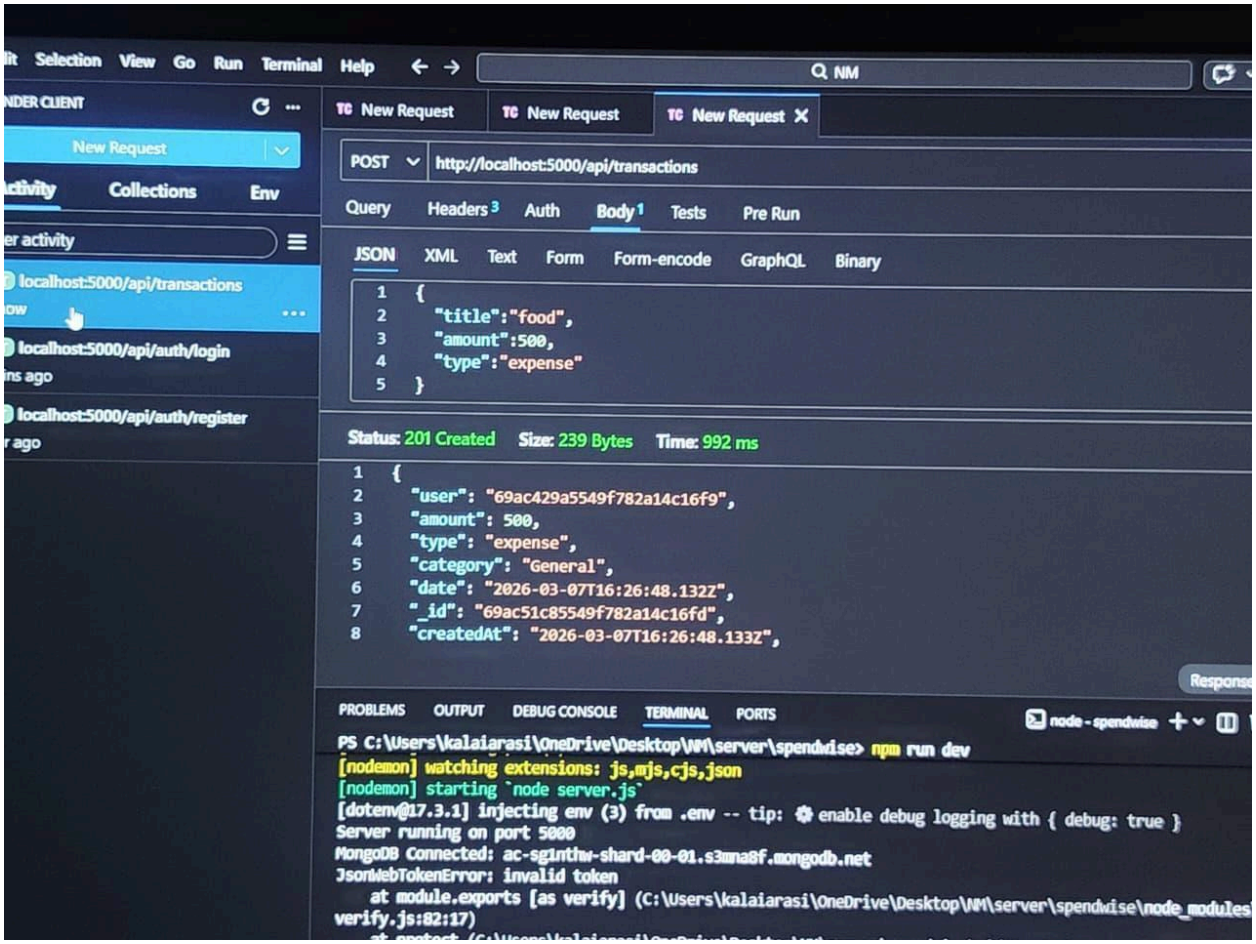
AI re
Generate

Suggested Acti
Build Workspa

Describe wha

26°C
Partly cloudy

Search



10.Conclusion

The SpendWise project is developed to help users easily track and manage their daily expenses. It provides a simple and efficient way to record spending, monitor financial activities, and analyze expense patterns. By using modern technologies, the system ensures secure data management and smooth user interaction.

Through features like expense submission, approval workflow, reporting, and analytics, SpendWise improves financial transparency and helps users make better financial decisions. The system also reduces manual work and saves time by automating expense tracking and reporting.